

Lott & Phillips (2021)

Hoofdstuk 9 – Strings, serialisatie en file paths

Kerninhoud

Hoofdstuk 9 behandelt hoe Python omgaat met:

- strings en tekstverwerking
- encoding en decoding (bytes ↔ tekst)
- regular expressions
- bestandspaden
- serialisatie en data-uitwisseling, met name via:
 - **pickle**
 - **JSON**
 - **CSV**

De focus ligt op correcte en consistente verwerking van data binnen applicaties.

Belangrijkste concepten (aangepast)

1. Strings en encoding

- Onderscheid tussen **str** (Unicode) en **bytes**
 - Correct gebruik van encoding (bijv. UTF-8) is essentieel bij I/O en communicatie
-

2. Regular expressions

- Toepassing voor patroonherkenning en validatie
 - Krachtig, maar gevoelig voor fouten bij complex gebruik
-

3. File paths

- Gebruik van gestandaardiseerde tools (bijv. **pathlib**)
- Belangrijk voor platformonafhankelijkheid en betrouwbaarheid

4. Serialisatie en dataformaten

- **pickle**
 - Serialiseert volledige Python-objecten
 - Ondersteunt complexe structuren
 - Gedrag kan worden aangepast (custom pickling)
- **JSON**
 - Tekstgebaseerd en interoperabel
 - Beperkt tot standaard datatypes
- **CSV**
 - Platte, tabelgebaseerde datarepresentatie
 - Veel gebruikt voor data-uitwisseling en eenvoudige opslag

👉 Belangrijk onderscheid:

- pickle → Python-specifiek en gedragrijk
- JSON/CSV → data-georiënteerd en interoperabel

Security-interpretatie

Het hoofdstuk zelf richt zich primair op **datarepresentatie en verwerking**, niet op beveiliging. Toch kunnen er vanuit een secure software development-perspectief relevante implicaties worden afgeleid:

- Deserialisatie met **pickle** kan risico's introduceren bij gebruik met onbetrouwbare input
- JSON en CSV beperken zich tot data, wat risico's op uitvoerbare payloads vermindert
- Onjuiste inputverwerking (strings, regex, encoding) kan leiden tot kwetsbaarheden zoals injectie

Deze koppeling sluit aan bij bekende kwetsbaarheden (bijv. onveilige deserialisatie), maar blijft een **interpretatie vanuit de modulecontext**, niet een expliciet doel van het hoofdstuk zelf.

Referentie (Harvard)

Lott, S.F. and Phillips, D. (2021) *Python Object-Oriented Programming*. 4th edn. Birmingham: Packt Publishing.

Junestam & Guigo (2014) – Open Crypto Audit Project: TrueCrypt Security Assessment

Context van het rapport

Het rapport maakt deel uit van het **Open Crypto Audit Project (OCAP)** en betreft een onafhankelijke beveiligingsanalyse van TrueCrypt, een destijds veelgebruikte open-source tool voor disk-encryptie.

De audit werd uitgevoerd naar aanleiding van toenemende zorgen over de betrouwbaarheid van cryptografische software en mogelijke achterdeuren, met name in het licht van de Snowden-onthullingen.

Het rapport vormt **fase 1 van de audit** en richt zich primair op architectuur, ontwerp en codekwaliteit van specifieke componenten.

Doel van de audit

Het doel was om te bepalen of er opzettelijke kwetsbaarheden (backdoors) aanwezig zijn en of er structurele zwaktes zijn in ontwerp of code.

Belangrijk: het betreft **geen formele verificatie**, maar een gerichte security-audit. Diepgaande cryptografische analyse (zoals random number generation, algoritme-implementaties en key management) viel expliciet buiten de scope van deze fase.

Scope en aanpak

De audit richt zich op:

- **TrueCrypt versie 7.1a**
- Kritieke componenten met verhoogde privileges:
 - bootloader
 - Windows kernel driver

De methodologie combineert:

- handmatige code review
 - beperkte statische analyse
-

Belangrijkste bevindingen

1. Geen bewijs voor backdoors

Er werd **geen bewijs gevonden voor opzettelijke achterdeuren of kwaadaardige code**.

Dit was een cruciale bevinding gezien de publieke zorgen over mogelijke staatsinmenging.

2. Medium-severity zwakte in key derivation

Het rapport identificeert een **kwetsbaarheid in PBKDF2 key derivation**, waarbij een relatief lage iteratiecount (1000 of 2000) werd gebruikt.

Dit verlaagt de weerstand tegen brute-force aanvallen. Tegelijkertijd geldt dat de complexiteit van de implementatie volledige verificatie bemoeilijkt.

3. Problemen met codekwaliteit

Belangrijke structurele zwaktes:

- Inconsistente code
- Beperkte documentatie
- Verouderde programmeerstijl

Gevolgen:

- verhoogde kans op fouten
- verminderde auditbaarheid
- moeilijk onderhoud

4. Beperkte modulariteit en hoge complexiteit

De codebase is:

- grotendeels monolithisch
- complex en weinig modulair opgebouwd

Gevolgen:

- lastig te testen
- moeilijk om security-eigenschappen te isoleren
- verhoogd risico op verborgen kwetsbaarheden

5. Gebruik van eigen implementaties en verouderde API's

TrueCrypt gebruikt in bepaalde gevallen:

- eigen implementaties in plaats van gestandaardiseerde libraries
- verouderde Windows API's

Risico:

- verhoogde kans op fouten
- verminderde externe verificatie
- afhankelijkheid van legacy-functionaliiteit

6. Build- en ontwikkelproces

Het rapport bekritiseert het ontwikkelproces, met name:

- beperkte transparantie
- complex en moeilijk te reproduceren buildproces
- beperkte documentatie

Gevolg:

- moeilijker om onafhankelijke verificatie uit te voeren

Nuance:

De audit signaleert deze tekortkomingen, maar positioneert ze als **verbeterpunten**, niet als formeel bewezen supply-chain kwetsbaarheid binnen deze fase.

Belangrijkste risico's (samengevat)

- Complexe en moeilijk te analyseren code
- Kwetsbaarheid in key derivation (PBKDF2)
- Beperkte onderhoudbaarheid
- Lage modulariteit
- Gebruik van custom en verouderde componenten
- Beperkte transparantie in ontwikkelproces

Security-interpretatie

Het rapport bevestigt een fundamenteel principe binnen cybersecurity:

Security is niet uitsluitend afhankelijk van sterke cryptografie, maar evenzeer van implementatiekwaliteit en ontwikkelproces.

Belangrijke inzichten:

- Implementatiefouten kunnen security ondermijnen
- Codekwaliteit is een directe security-factor
- Complexiteit vergroot het risico op kwetsbaarheden
- Transparantie ondersteunt auditbaarheid en vertrouwen

Referentie (Harvard)

Junestam, A. and Guigo, N. (2014) *Open Crypto Audit Project: TrueCrypt Security Assessment*. iSEC Partners.

TutorialPoint (2020) – Cryptography with Python Tutorial

Context van de bron

De tutorial van TutorialPoint biedt een praktische introductie tot cryptografie in Python. De nadruk ligt op het uitleggen van basisconcepten en het demonstreren van implementaties via codevoorbeelden, niet op formele verificatie of diepgaande dreigingsanalyse.

Het betreft een educatieve handleiding gericht op ontwikkelaars die cryptografische technieken willen begrijpen en toepassen in Python.

Doel van de tutorial

Het doel is om inzicht te geven in hoe cryptografische technieken functioneren en hoe deze in Python geïmplementeerd kunnen worden.

De tutorial combineert:

- uitleg van basisprincipes
- praktische codevoorbeelden
- demonstraties van zowel eenvoudige als meer geavanceerde technieken

De nadruk ligt op toepasbaarheid en begrip, niet op formele security-evaluatie.

Structuur en inhoud

De tutorial behandelt een breed scala aan onderwerpen, met een opbouw van eenvoudig naar complexer gebruik:

- Klassieke cryptografische voorbeelden (zoals Caesar cipher en Vigenère) ter illustratie van basisprincipes
- Symmetrische en asymmetrische cryptografie
- RSA-sleutelgeneratie, encryptie en decryptie
- Hashing via Python-modules (bijv. `hashlib`)
- Gebruik van bestaande libraries, waaronder het `cryptography`-pakket
- Bestandsencryptie en eenvoudige data-transformaties (zoals Base64 en XOR)

Klassieke cryptografie wordt primair gebruikt als didactisch hulpmiddel, terwijl de nadruk ligt op moderne toepassingen en Python-implementaties.

Belangrijkste concepten

Symmetrische encryptie

De tutorial laat zien hoe encryptie werkt met één gedeelde sleutel voor zowel encryptie als decryptie. De voorbeelden zijn bedoeld om de werking van vertrouwelijkheid inzichtelijk te maken.

Asymmetrische encryptie (RSA)

Een belangrijk onderdeel richt zich op RSA:

- genereren van publieke en private sleutels
- encryptie en decryptie
- demonstraties van conceptuele zwaktes

Dit maakt inzichtelijk hoe asymmetrische cryptografie functioneert in de praktijk.

Hashing

Hashing wordt geïntroduceerd via Python-modules zoals `hashlib`:

- toepassing voor integriteitscontrole
- illustratie van het verschil tussen hashing en encryptie

De nadruk ligt op basisgebruik, niet op diepgaande analyse van specifieke algoritmes.

Python-implementaties

De tutorial maakt gebruik van:

- standaard Python-modules
- bestaande cryptografische libraries (zoals `cryptography`)
- eenvoudige voorbeeldimplementaties (met name bij klassieke ciphers)

Dit ondersteunt begrip van onderliggende principes, maar vereist voorzichtigheid bij toepassing in productieomgevingen.

Praktische toepassingen

Voorbeelden zoals:

- bestandsencryptie
- Base64-encoding
- XOR-bewerkingen

maken de concepten direct toepasbaar in eenvoudige scripts.

Beperkingen van de tutorial

De tutorial kent duidelijke beperkingen:

- De nadruk ligt op uitleg en implementatie, niet op formele verificatie of diepgaande dreigingsanalyse
- Onderwerpen zoals:
 - dreigingsmodellen
 - attack vectors
 - secure software developmentworden niet expliciet uitgewerkt of staan niet centraal
- Sleutelbeheer komt slechts op basisniveau aan bod
- Gebruik van eenvoudige of verouderde technieken zonder uitgebreide securitycontext
- Educatieve voorbeelden zijn niet bedoeld als productieklare security-oplossingen

Security-interpretatie

De tutorial laat zien dat cryptografie in Python toegankelijk is, maar ook dat onjuist of oppervlakkig gebruik kan leiden tot kwetsbare implementaties.

Belangrijke inzichten:

- Begrip van cryptografische principes is noodzakelijk, maar onvoldoende voor veilige systemen
- Het gebruik van bestaande, goed geteste libraries is essentieel
- Implementatiecontext en correcte toepassing bepalen de uiteindelijke security
- Educatieve voorbeelden mogen niet direct als veilige oplossingen worden geïnterpreteerd

Referentie (Harvard)

TutorialPoint (2020) *Cryptography with Python Tutorial*. Available at:
https://www.tutorialspoint.com/cryptography_with_python/index.htm